

BugSense AI

Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance

Project Handover, Features, Setup, Run & Demo Guide

Frontend: React + Vite + Tailwind CSS | Backend: FastAPI + SQLite

1. Project Overview

BugSense AI is an intelligent bug diagnosis and engineering support platform. It allows users to submit bug reports, source code, logs, stack traces, and supporting files, then run a multi-agent diagnosis pipeline that produces structured engineering guidance, historical matches, root-cause information, remediation suggestions, and a 0-100 risk score.

Primary objective: reduce manual bug-triage effort and reuse historical debugging knowledge so that recurring problems can be identified and resolved faster.

2. Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	User interface and client-side interaction
Backend	FastAPI, Python	REST API and application logic
Database	SQLite + SQLAlchemy	Users, bugs, analysis, comments, events and history
Authentication	JWT + bcrypt	Login and role-based access
AI / Analysis	Multi-agent diagnosis pipeline	Triage, log parsing, duplicates, root cause and remediation
Historical Retrieval	Similarity / knowledge-base retrieval	Reuse of resolved historical bugs and fixes
Reports	PDF generation	Exportable diagnostic reports
Charts	Chart.js / react-chartjs-2	Analytics and engineering insight visualizations

3. Main Features

- New Diagnosis Workspace:** Submit a structured bug report with title, description, project, component/module, category, severity and priority.
- Raw Source-Code Input:** Switch to Source Code mode and submit code directly for diagnostic context.
- Multi-language Log Parsing:** Parses common Python, JavaScript/Node/TypeScript, Java, C/C++, Go, Rust and generic error logs.

- **Supporting File Upload:** Accepts common text/log/code files in the diagnosis workflow; attachments can also be added later to a case.
- **Five-Agent AI Pipeline:** Triage Agent, Log Intelligence, Similarity/Duplicate Detection, Root Cause Agent and Remediation Agent.
- **AI Bug Risk Score:** Generates a 0-100 risk score using severity, priority, root-cause confidence, recurrence/similarity, exception evidence and failure localization.
- **Duplicate / Similar Bug Detection:** Compares the current case against historical bugs and shows similarity percentages.
- **Historical Knowledge Vault:** Resolved bugs with resolution knowledge become reusable evidence for future diagnosis.
- **Root Cause Analysis:** Produces a likely root cause with a confidence score and historical grounding when relevant evidence exists.
- **Fix Recommendation:** Suggests remediation, estimated fix time, best practices and prevention tips.
- **Case History:** Search and filter previous diagnostic cases by severity and status.
- **Lifecycle Management:** Move cases through Open, In Progress, Resolved and Closed states.
- **Resolution Knowledge:** Store verified resolution notes so future cases can learn from successful fixes.
- **Attachments and OCR:** Upload evidence; image text can be extracted when OCR is available and appended to runtime evidence.
- **Comments / Investigation Notes:** Collaborate by adding notes to a case.
- **Audit Timeline:** Tracks creation, status updates, comments, attachments, analysis runs and other case events.
- **Analytics & Project Health:** Shows total/open/resolved/critical cases, 30-day trend, severity/status/category charts, health score and team performance.
- **PDF Diagnostic Report:** Download a PDF report for a bug case.
- **BugSense Copilot:** In-app assistant that can answer questions about bugs, critical cases, similar cases and project health.
- **Role-based Administration:** Admin users can view workspace users, assign roles and remove accounts while protected from self-removal/self-role changes.
- **Dark / Light Theme:** Theme toggle available from the top bar.

4. Multi-Agent Diagnosis Workflow

1. Triage Agent - Predicts severity, priority and category and returns confidence/reasoning.

2. Log Intelligence - Extracts exception type, failure file, line number, function and a log summary.

3. Similarity Agent - Searches historical bugs and returns the closest matching cases with similarity percentages.

4. Root Cause Agent - Produces the probable root cause and confidence, using historical resolved cases where available.

5. Remediation Agent - Suggests a fix, best practices, prevention guidance and estimated fix time.

6. Risk Engine - Combines evidence from the above stages into an overall 0-100 bug risk score and risk level.

5. Project Folder and Important Paths

If the ZIP is extracted on Windows, open the project root that contains both the backend and frontend folders. Example structure:

```
smart-bug-analyzer/  
├── backend/  
├── frontend/  
├── uploads/  
└── ...
```

Important: Run the backend and frontend in two separate terminals.

6. Requirements Before Running

- Python 3.12 is recommended for the backend environment.
- Node.js and npm must be installed for the frontend.
- Use VS Code, PowerShell, Command Prompt, or another terminal.
- Keep internet access available if the configured AI provider requires it.
- If a .env.example file is provided, create the required .env file and fill any API keys/configuration values expected by the project.

7. How to Run the Backend

Step 1 - Open the backend folder

```
cd path\to\smart-bug-analyzer\backend
```

Step 2 - Create a Python 3.12 virtual environment (first run only)

```
py -3.12 -m venv .venv
```

Step 3 - Activate the environment

```
.venv\Scripts\Activate.ps1
```

If PowerShell activation is restricted, use Command Prompt activation instead:

```
.venv\Scripts\activate.bat
```

Step 4 - Install backend dependencies (first run only)

```
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

Use python -m pip rather than calling pip.exe directly if Windows Application Control blocks pip.

Step 5 - Start FastAPI

```
python -m uvicorn app.main:app
```

Backend address: **http://127.0.0.1:8000**

Keep this terminal open while using the application.

8. How to Run the Frontend

Step 1 - Open a second terminal

Step 2 - Go to the frontend folder

```
cd path\to\smart-bug-analyzer\frontend
```

Step 3 - Install packages (first run only)

```
npm install
```

Step 4 - Start Vite

```
npm run dev
```

Open the URL printed by Vite. The normal local address is: **http://localhost:5173**

9. Login and User Access

- Use a registered account from the project database, or create an account through the application if registration is enabled.
- Do not assume a default password unless one is explicitly provided with the shared ZIP.
- Supported roles include Developer, Team Lead, QA Engineer, Engineering Manager and Admin.
- Admin privileges are granted through the Admin / Control Center rather than being self-selected by a normal user.

10. Recommended Demo Flow

1. Log in and open Overview to show the dashboard and current engineering metrics.
2. Open New Diagnosis.
3. Click Load demo case, or enter a fresh test case manually.
4. Create the diagnostic case.
5. Open the created case and click Run AI diagnosis / Re-run AI analysis.
6. Show the AI Risk Intelligence score and risk factors.
7. Show Triage, Log Intelligence, Historical Similarity, Root Cause and Remediation results.
8. Show the runtime failure point (file, line and function) if parsed.
9. Open a similar historical case from the similarity results.
10. Mark a case Resolved and save resolution knowledge to demonstrate historical learning.
11. Open Knowledge Vault to show resolved cases becoming reusable AI knowledge.
12. Open Insights to show analytics, project health and charts.
13. Download the PDF report from a case.
14. Open BugSense Copilot and ask a question such as “List critical bugs” or “What is the health score?”.
15. Show Case History search/filtering and the lifecycle controls.

11. Demo Test Case

Field	Sample Value
Title	ZeroDivisionError in calculation service
Description	The calculation service crashes when a division operation is

Field	Sample Value
	performed with zero as the denominator. The issue occurs during runtime and prevents the calculation request from completing successfully.
Project	BugSense Demo
Component / Module	calculation-service
Category	Backend
Severity	High
Priority	P1

Stack trace:

```
Traceback (most recent call last):
  File "calculator.py", line 24, in divide
    result = a / b
ZeroDivisionError: division by zero
```

12. Expected Demo Output

- The case is stored and visible in Case History.
- Log Intelligence should identify ZeroDivisionError and, when the trace is formatted as above, calculator.py line 24 in divide().
- Triage should produce an AI severity/priority/category prediction with confidence.
- Similarity Detection should list historical matches if similar bugs exist in the database.
- Root Cause should provide a likely cause and confidence.
- Remediation should provide fix guidance and prevention recommendations.
- The Risk Engine should return an overall 0-100 score and a risk level such as Minimal, Low, Medium, High or Critical.

13. Historical Learning / Knowledge Workflow

Historical learning becomes more useful after resolved cases have valid resolution notes. Recommended process:

16. Open a previously solved case.
17. Set its status to Resolved.
18. Enter clear resolution knowledge describing the verified root cause and successful fix.
19. Save the resolution knowledge.
20. Open Knowledge Vault and verify the case appears there.
21. Create or analyze a similar new case and compare the Similarity and Root Cause results.

14. Troubleshooting

Problem	Likely Cause	Fix
uvicorn is not recognized	Uvicorn not available in the active environment	Activate .venv and run: python -m pip install -r requirements.txt, then python -m uvicorn app.main:app
pip.exe is blocked	Windows Application Control / execution policy	Use python -m pip instead of pip.
Frontend cannot connect to backend	Backend is not running or configured API address is wrong	Start FastAPI at 127.0.0.1:8000 and verify frontend environment configuration.
Page does not reflect frontend changes	Browser/Vite cache or unsaved file	Save the file, allow Vite hot reload, then

Problem	Likely Cause	Fix
		refresh the browser.
Backend code change does not appear	Uvicorn was started without --reload	Stop the backend with Ctrl+C and start it again.
AI analysis fails	AI/API configuration unavailable, backend error or missing data	Check the backend terminal and required environment variables/API key.
No useful historical root cause	No similar resolved case with resolution notes exists	Resolve a matching old case, save clear resolution knowledge, then analyze the new case again.
Knowledge Vault is empty	No resolved bugs have saved resolution notes	Resolve a bug and save resolution knowledge.
Sign-out/menu overlap	Frontend layering issue	Use the current redesigned Topbar included in the final project ZIP.

15. Quick Start Summary

```

BACKEND TERMINAL
cd backend
.venv\Scripts\Activate.ps1
python -m uvicorn app.main:app

FRONTEND TERMINAL
cd frontend
npm run dev

OPEN
http://localhost:5173

```